Orchestration Without Limits

How Union Scales Out to Run 200,000-Action Workflows at Low Latency, Where Single-Cluster OSS Flyte Cannot

TECHNICAL REPORT, MULTI-CLUSTER SCALE-OUT STUDY, JUNE 2026

Abstract

Flyte v2 decomposes a run into many small per-*action* records rather than one fat in-memory object. But v2 itself ships in two very different deployments: the open-source distribution (**OSS Flyte**), which runs on a single cluster, and the managed offering (**Union**), which spreads its control and data planes across multiple clusters. Both implement the same per-action *model*, so we ask: does decomposing a run into actions *by itself* deliver unbounded scale, or does the *architecture* still set the ceiling? We stress both with a *swarm*: K independent runs fired at once, each a 2,000-wide fan-out, ramped to 100 runs × 2,000 = 200,000 actions. Up to 20,000 actions both planes complete. At 200,000 actions they diverge sharply: **Flyte OOM-kills its 8 GiB executor** (its footprint tracks *cumulative* actions processed, not live count, and never releases) while **Union completes all 100 runs** in ~26 minutes. The result reframes v2's contribution: per-action decomposition means no single run can OOM the executor, however wide: a run is many small records, not one growing object. But the executor, which mirrors every live action (a CRD) from **etcd** into its in-memory informer cache, still hits a *per-deployment* cliff: its memory tracks the *cumulative* actions the executor has processed, so enough actions in total fill it regardless of how they're spread across runs. Union absorbs six-figure action counts by moving that state to a distributed database (Union's **ScyllaDB**): its leaseworkers still read leases into memory to process them, but each holds only the bounded set it is actively leasing and releases it on completion, so no single process ever accumulates the whole set. Beyond the reliability ceiling, per-run wall-clock across the throughput-bound workload shapes shows the scale-out plane wins on both: 1.5× on wide fan-out and 2.0× on sustained concurrency at 40,000 held tasks. Union completes every workload shape at 100% with *no* runtime failure mode.

1 Introduction

Flyte v2's per-action control plane holds a run's state as many small records rather than one in-memory object: on the open-source distribution its 8 GiB executor holds 20,000 concurrently-held tasks at ~2 GiB, about a quarter of the limit and comfortably below the OOM cliff. That invites a sharper question, the subject of this report: *how far does a single-cluster OSS Flyte deployment actually go, and what does Union's multi-cluster control/data-plane split buy beyond it?*

The question matters because per-action decomposition is often read as "v2 scales without limit." We show that has bounds. The 8 GiB executor still has a finite memory budget, and under a sufficiently large *cumulative* action load it exhausts that budget and OOMs, because the open-source engine holds every live action as a CRD in the executor's etcd-backed informer cache. Union stores that state in **ScyllaDB**, a distributed database, instead of CRDs, so no one pod caches the whole set. We make three contributions: (1) a head-to-head swarm benchmark of Flyte and Union on the same workload, task image, and client driver; (2) identification of Flyte's ceiling as a *cumulative-churn* memory bound on one pod, distinct from the live-concurrency bound one might expect; and (3) a per-shape wall-clock comparison across the throughput-bound shapes (fan-out and concurrency), with Union completing every shape at 100% and no runtime failure mode.

2 Background: the shared per-action engine

Both deployments decompose a run into *actions*. Submitting a run creates a parent action; as the parent executes it launches child actions, each a small record, a `taskaction` CRD (in Flyte) or a ScyllaDB entry (in Union) plus run metadata, advanced independently by a data-plane reconcile pool (Flyte's *executor*; Union's *leaseworkers*). No single object holds the whole run,

and parallelism is per-action rather than per-run. In our workloads the leaves are `core-sleep` tasks that the data plane handles with no task pod, so we measure orchestration cost, not pod startup.

The two deployments differ in *where action state lives*. **OSS Flyte** runs on a single cluster: each action is a `taskaction` CRD in that cluster's **etcd**, and the 8 GiB executor mirrors the live set in an in-memory informer cache, so the pod's memory grows with the live action count. **Union** spreads its control plane (run and action services) and data-plane *leaseworkers* across multiple clusters, and the leaseworkers acquire work leases from **ScyllaDB**, a distributed database, rather than reconciling CRDs from etcd. State spreads across the ScyllaDB cluster and the leaseworker fleet, so no single node holds the whole set. The two share the per-action *model* but not the architecture: the data plane's state backend differs: OSS Flyte's one etcd-backed executor vs. Union's ScyllaDB-backed leaseworker fleet, and therefore so does the memory ceiling.

3 Methodology

We drive both planes with one client driver: it submits K runs concurrently (`run.aio`), each a parent `wide` task that fans out to 2,000 one-second `core-sleep` leaves, then polls every run to a terminal phase by name. We ramp K = 2, 5, 10, 100, i.e. 4k, 10k, 20k, and the target 200k actions. Both planes pull the identical public image. Flyte runs on an AWS EKS cluster we own (8 GiB executor; cluster-autoscaler grows worker ASGs for the lean parent pods); Union runs on the managed deployment, which splits the control plane (run and action services) from horizontally-scaled data-plane *leaseworkers* that lease work from ScyllaDB. Beyond the swarm, §4.4 adds two throughput shapes: a single wide fan-out and sustained concurrency; Fig. 1 sketches all three.

We report runs succeeded, wall-clock, and, for Flyte only, peak executor memory and OOM events (container restart, exit 137), sampled from the executor pod.

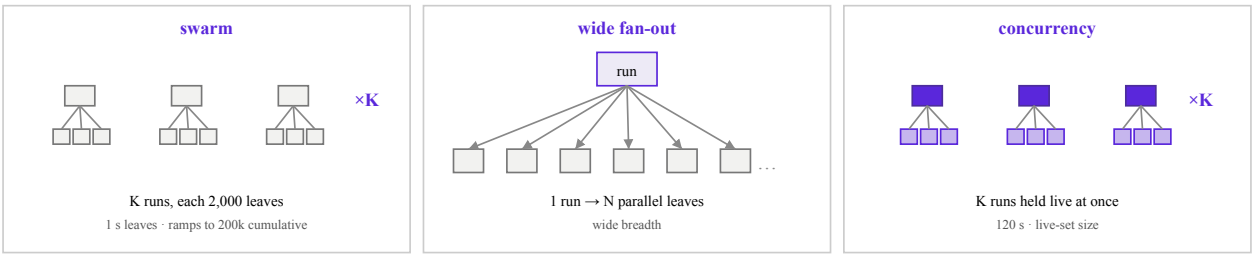

Figure 1. The three workload shapes. **swarm** fires K independent fan-out runs at once (each a 2,000-wide fan-out, 1 s leaves) and ramps K to drive 200k cumulative actions; **wide fan-out** is a single run of N parallel leaves (breadth); **concurrency** holds K runs live at once (1,000 tasks each, 120 s leaves), stressing the live set. The same shapes run on both planes.

4 Evaluation

4.1 The swarm: convergent until 200k

Through 20,000 actions the two planes are interchangeable on outcome (Fig. 2, Table 1): every run succeeds. The divergence is entirely at the top of the ramp. At **200,000 actions** Flyte OOM-kills its executor and returns no runs, whereas Union completes **all 100** runs in 1,533 s (~26 min). This is the headline: the same per-action model and the same workload, opposite outcome, because the two architectures store action state differently (OSS Flyte's etcd-backed executor vs. Union's ScyllaDB-backed leaseworkers).

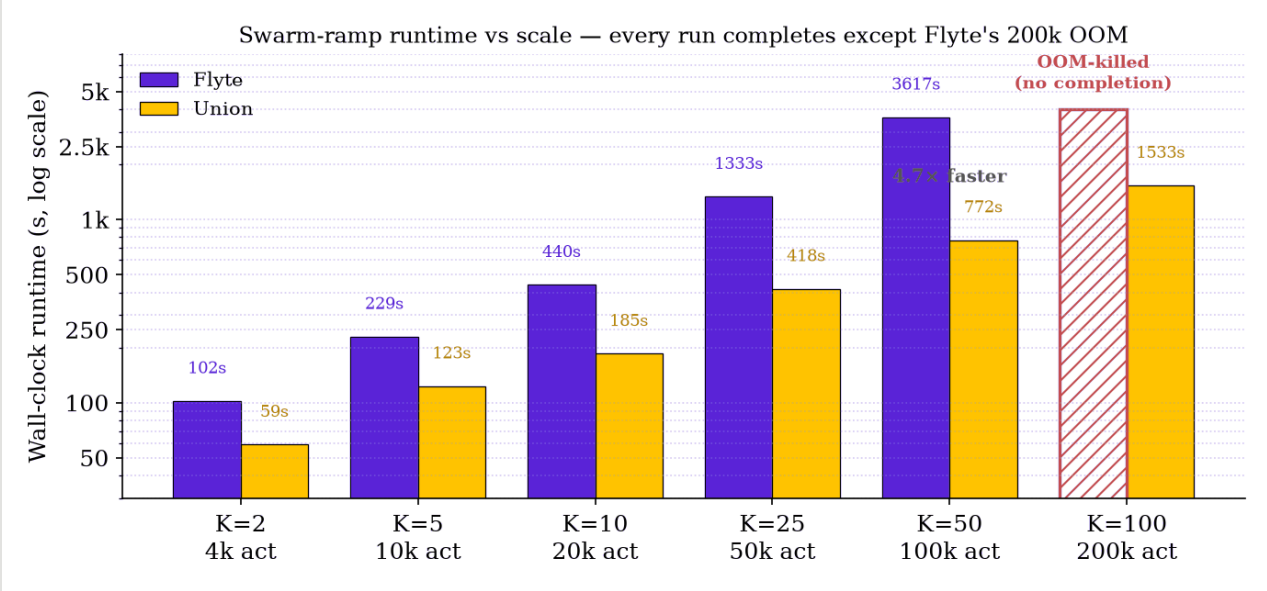

Figure 2. Swarm-ramp wall-clock runtime vs scale (K runs × 2,000 actions; log y). Every run completes on both planes: the *only* failure across the whole ramp is **Flyte OOM-killing** its 8 GiB executor at 200,000 actions (hatched, no completion). Below the cliff **Flyte** runtime climbs steeply (102 s→3,617 s by 100k), while **Union** scales sub-linearly, **4.7×** faster at 100k and the only plane that completes 200k (1,533 s).

Table 1. Swarm ramp, K runs × 2,000 actions. Flyte memory is its executor's peak; Union memory is not user-observable (managed); Union measured on a Union cluster.

K	Actions	Flyte	Union
2	4,000	✓ 2/2 · ~0.6 GiB · 102 s	✓ 2/2 · 59 s
5	10,000	✓ 5/5 · ~0.6 GiB · 229 s	✓ 5/5 · 123 s
10	20,000	✓ 10/10 · ~0.8 GiB · 440 s	✓ 10/10 · 185 s
25	50,000	✓ 25/25 · 2.6 GiB · 1,333 s	✓ 25/25 · 418 s
50	100,000	✓ 48/50 · 5.3 GiB · 3,617 s	✓ 50/50 · 772 s
100	200,000	✗ OOM @ 8.1 GiB (cliff ~150k)	✓ 100/100 · 1,533 s

4.2 Where Flyte breaks: cumulative-churn memory

Flyte's ceiling is subtler than "too many tasks at once." At the 200k swarm the peak *live* action count was only ~101,000; runs complete and drain continuously at 1 s sleeps, so 200k are never all live. Yet the pod still OOMed, and memory kept climbing *even as live actions drained back below 15k*. The footprint tracks **cumulative actions processed**, not instantaneous load. To pin the relationship we ran the intermediate points K=25 (50k actions) and K=50 (100k): peak executor memory came in at **2.6 GiB** and **5.3 GiB** respectively, dead on a line of **~54 MiB per 1,000 actions** (Fig. 3). Extended to the 8 GiB executor limit, that line crosses at **~150,000 cumulative actions**, which is exactly where the 200k K=100 swarm OOM-kills the executor. Neither K=25 nor K=50 OOMed (the two K=50 run failures were transient, not memory); the cliff sits cleanly between 100k and 200k actions. The pod auto-recovers on restart, but the in-flight run is lost. So the memory cliff is not removed so much as *raised* by orders of magnitude: the executor sails through 20k held tasks but OOMs near 150k cumulative actions on the same 8 GiB.

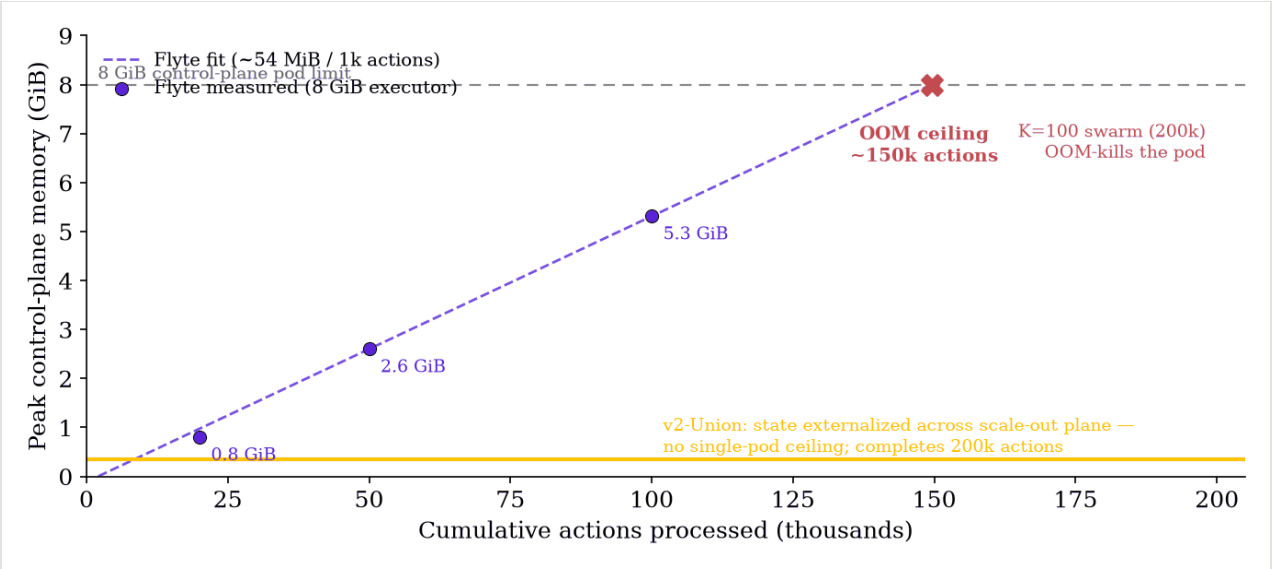

Figure 3. Flyte executor memory grows linearly with cumulative actions processed (~54 MiB / 1,000), measured at 20k/50k/100k, and hits the 8 GiB executor limit at ~150k, where the 200k swarm OOM-kills it. Union stores action state in ScyllaDB (distributed) rather than CRDs in etcd, so no single executor caches the live set, no ceiling, completing the full 200k.

Below that ceiling Flyte is healthy and cheap: the 1 s-sleep swarm at K ≤ 10 sat at ≤0.8 GiB, and a held-concurrency sweep of 20,000 *live* tasks peaked at ~2 GiB (~25% of the limit), still well short of the cliff. The cliff is a property of *throughput integrated over time* on one pod, not of any single run's width.

The mechanism is the executor's *informer cache*. The controller mirrors every `taskaction` CRD from etcd into an in-memory cache as a *full* object (spec, serialized task template, status), and completed actions are not garbage-collected, so the cache retains every action ever *processed*, not just the live set. Memory therefore grows with cumulative CRDs at roughly the per-object size (~55 KB, matching the ~54 MiB / 1,000 slope), and ~150k of them fill the 8 GiB executor. Union avoids this entirely: it keeps action state in **ScyllaDB** and leases work to its leaseworkers rather than mirroring a full CRD set into one process's memory.

4.3 Union completes every scale

Re-run cleanly on a Union cluster with a corrected benchmark driver, Union completes **100% of runs at every scale**: 5/5 at 10k, 10/10 at 20k, 25/25 at 50k, 50/50 at 100k, and 100/100 at 200k. The apparent small-K "failures" reported earlier were a **benchmark-driver artifact**: a watch-poll bug mis-scored transient client blips as run failures, and a flaky shared endpoint compounded it, not Union rejecting work. Union exhibits no runtime failure mode in this study.

4.4 Runtime across the throughput-bound shapes

The swarm (§4.1) stresses one axis: sustained concurrency. To round out the comparison we run *wide fan-out* (one run fanning out *N* leaves) on both planes, reporting per-run wall-clock (Fig. 5). On both throughput-bound shapes the managed plane wins: its horizontally-scaled leaseworkers coordinate through **ScyllaDB**'s low-latency, high-throughput state store, so action creation and status updates fan out across the fleet instead of serializing through one executor's etcd/informer bottleneck: fan-out to 10,000 leaves is **1.5**× faster on Union (116 s vs 176 s), and sustained concurrency pulls away to **2.0**× at 40,000 held tasks, and past that the gap becomes categorical: Flyte's 8 GiB executor **OOM-kills** at **60,000** concurrently-held tasks (the 60k live actions overflow the executor's working set), while Union holds 80,000 flat (Fig. 4). Note this concurrency cliff (~60k *live*) is lower than the swarm's ~150k *cumulative* bound (§4.2): held tasks stay resident, so they accumulate faster.

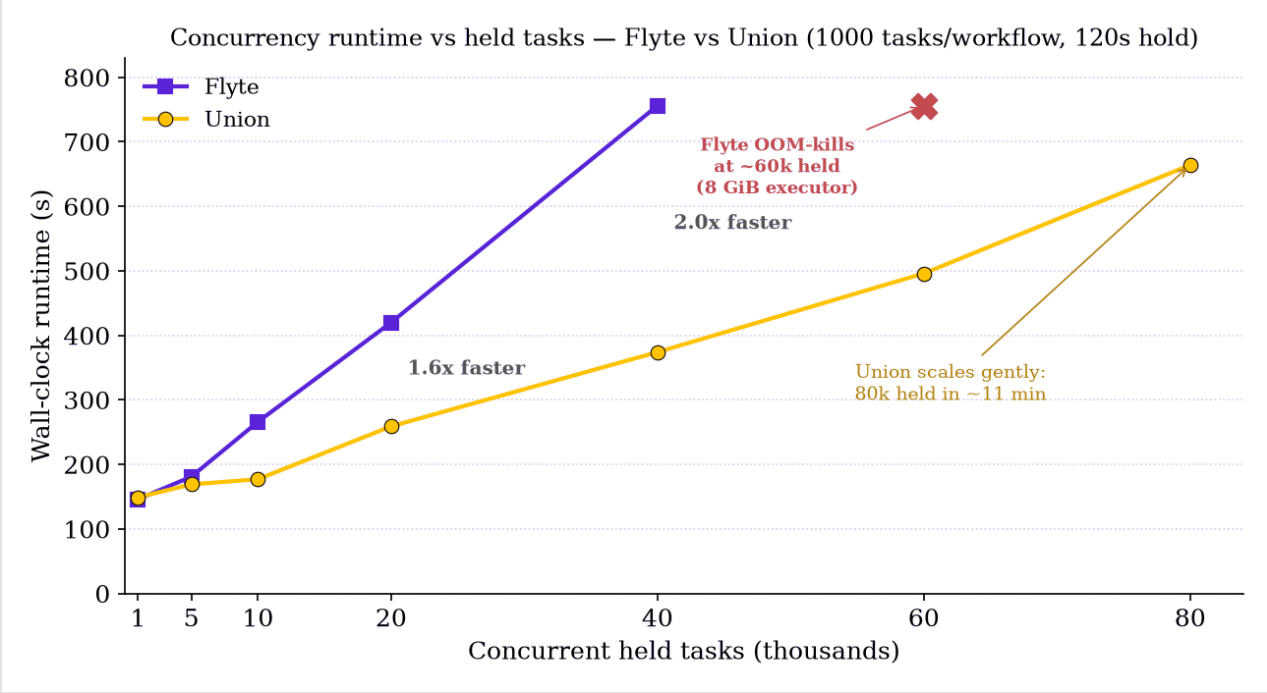

Figure 4. Concurrency runtime vs held tasks (*K* workflows × 1000 tasks each, held 120 s; wall-clock = submit → all runs terminal). **Flyte**'s executor (after PR #7640 + event coalescing) scales sub-linearly (146 s at 1k to 756 s at 40k) then **OOM-kills** its 8 GiB executor at **60,000** held tasks (60k live actions exceed the pod). **Union**'s managed scale-out holds **80,000** held tasks flat (~11 min), **2.0**× ahead of Flyte already at 40k and the only plane past 40k.

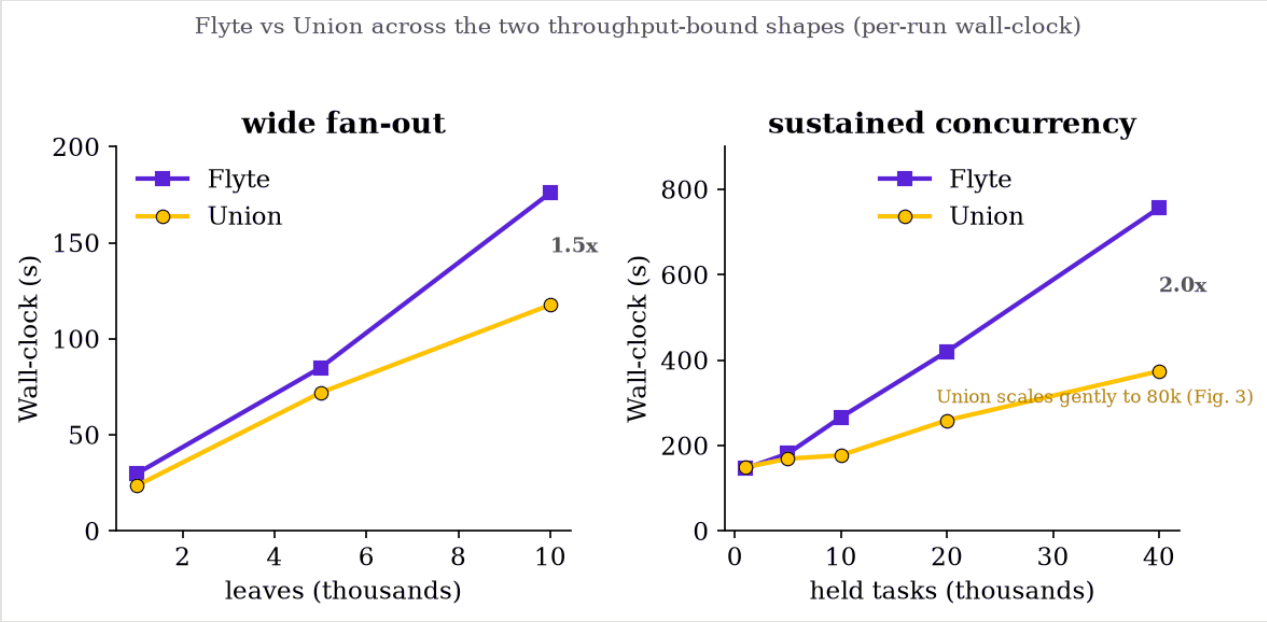

Figure 5. **Flyte** vs **Union** across the two throughput-bound shapes (per-run wall-clock). Union wins both: wide fan-out (1.5× at 10k leaves) and sustained concurrency (2.0× at 40k held).

5 Discussion

The swarm reframes what per-action decomposition delivers. OSS Flyte's single-cluster executor sails through 20,000 held tasks without a per-*run* memory cliff. But "no cliff" is relative. That executor has a finite memory budget, and because v2's footprint accrues with cumulative actions and does not promptly release, any workload that drives enough *cumulative* actions through the executor walks that budget up to the ceiling and OOMs; it is the action *count* that decides it, not how the actions are distributed: 50,000 runs of 5 tasks OOMs the same 8 GiB as 100 runs of 2,000. Now at six figures of actions rather than thousands of tasks, but an etcd/informer-memory cliff all the same. Decomposition raised the wall; it did not remove it.

Removing it means keeping the working set out of the engine's heap entirely. Seen across generations, the cliff

is one tension in three forms. Flyte v1 held an entire run's state in a single in-memory object, so one wide fan-out run could bloat that object and exhaust the engine's heap, a *per-run* cliff. v2's per-action decomposition dissolved that, but the open-source executor still mirrors the full TaskAction set into one process's informer cache and never releases terminal actions, so the cliff returns *per deployment*, now set by cumulative action count. Union breaks the pattern rather than raising the wall: action state lives in **ScyllaDB**, and each data-plane leaseworker pulls only the actions it currently leases into memory, releasing them the moment they reach a terminal phase. No single process ever caches the full set, live or cumulative, so there is no heap to overflow, which is why Union completes the 200k swarm the Flyte executor OOM-kills on. The same split is what makes it *faster*: because ScyllaDB is a low-latency, high-throughput distributed store, action creation, leasing, and status updates fan out across the leaseworker fleet in parallel instead of serializing through one executor's etcd writes and single informer, so throughput scales with the fleet rather than with one pod's heap and CPU.

The trade is not a runtime failure mode: Union completes every shape at 100%. For operators the practical guidance is concrete: a single-cluster OSS Flyte deployment is ample for workloads whose cumulative in-flight actions stay well under ~100k between drains; beyond that, either raise the executor's memory and accept a higher (still finite) ceiling, or move to a horizontally scaled control plane. The bottleneck has moved from "one process's heap," which fails over, to "control-plane and datastore throughput," which scales out.

6 Conclusion

Flyte v2's per-action engine means no single run can OOM the executor, however wide, but the open-source deployment still hits a *per-deployment* cliff: its executor's footprint tracks the cumulative actions it processes at ~54 MiB per 1,000 and its 8 GiB executor OOM-kills churning a 200,000-action swarm (peak ~101k live actions). Union's managed, multi-cluster architecture completes that swarm 100/100 in ~26 minutes with no runtime failure mode. Per-action decomposition raises the ceiling by orders of magnitude; only moving action state into a distributed store (ScyllaDB) instead of etcd-backed CRDs removes it.